

Homework 4

Kun Yuan

Dec 25, 2025

1 Problems

A1. Consider a Markov Decision Process (MDP) defined by the tuple $(\mathcal{S}, \mathcal{A}, P, \mathcal{R}, \gamma)$.

- Prove the following conclusions for the State-Value Function and Action-Value Function:

$$q_{\pi}(s, a) = \sum_{r \in \mathcal{R}} p(r \mid s, a) r + \gamma \sum_{s' \in \mathcal{S}} p(s' \mid s, a) v_{\pi}(s')$$

$$v_{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) q_{\pi}(s, a).$$

- Prove the bellman equations for v_{π} and q_{π} :

$$v_{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s' \in \mathcal{S}, r \in \mathcal{R}} p(s', r \mid s, a) [r + \gamma v_{\pi}(s')],$$

$$q_{\pi}(s, a) = \sum_{s' \in \mathcal{S}, r \in \mathcal{R}} p(s', r \mid s, a) \left[r + \gamma \sum_{a' \in \mathcal{A}} \pi(a' \mid s') q_{\pi}(s', a') \right].$$

A2. Consider a finite-horizon MDP $(\mathcal{S}, \mathcal{A}, P, R, H)$, where H is a fixed termination time (horizon). Define $\tau = (s_1, a_1, s_2, \dots, a_{H-1}, s_H)$ as a trajectory (state-action sequence) generated by executing policy π starting from state s_1 , where $a_t \sim \pi(\cdot \mid s_t)$. Compute the probability $p_{\pi}(\tau)$ for the trajectory τ .

A3. Consider a simple MDP with 3 states $\mathcal{S} = \{s_1, s_2, s_3\}$ and 3 actions $\mathcal{A} = \{a_1, a_2, a_3\}$.

- **Discount factor:** $\gamma = 0.9$.
- **Policy:** The following table gives the values of each $\pi(a \mid s)$:

$\pi(a \mid s)$	s_1	s_2	s_3
a_1	1/2	1/4	1/4
a_2	1/4	1/2	1/4
a_3	1/4	1/4	1/2

- **Reward:** The reward depends on the state occupied. $R(s) = 1$ if $s = s_3$, and 0 otherwise.
- **Goal:** For the two cases below, determine:
 - (a) The **Stationary Distribution** $d(s)$ for $s \in \{s_1, s_2, s_3\}$ of the induced Markov Chain.
 - (b) The **Value Function** $v_{\pi}(s)$ for all states.Calculators are allowed to be used to obtain the results.

Case 1: Deterministic Transition

The state transition probability $p(s' \mid s, a)$ is deterministic and depends only on the action taken, regardless of the current state. The transitions are defined for $i = 1, 2, 3$ as:

$$(s_i, a_1) \rightarrow s_{i-1}, (s_i, a_2) \rightarrow s_i, (s_i, a_3) \rightarrow s_{i+1},$$

with the boundary notations $s_0 = s_3, s_4 = s_1$.

(Mathematically: $p(s' \mid s, a) = 1$ if the target matches the action rule, else 0).

Case 2: Stochastic Distribution

The state transition $p(s'|s, a)$ is stochastic. Regardless of the current state s or action a , the next state follows a fixed probability distribution. The transitions are defined for $i = 1, 2, 3$ as:

$$p(s_{i-1}|s_i, a_1) = \frac{1}{2}, p(s_i|s_i, a_1) = \frac{1}{2}, p(s_i|s_i, a_2) = 1, p(s_{i+1}|s_i, a_3) = \frac{1}{2}, p(s_i|s_i, a_3) = \frac{1}{2},$$

with the boundary notations $s_0 = s_3, s_4 = s_1$.

A4. Consider the **Multi-Head Attention (MHA)**. We aim to derive the computational cost (in terms of floating-point operations, FLOPs) for the core matrix multiplication operations within the attention mechanism.

Let the input hyperparameters be defined as follows:

- s : The sequence length (number of tokens).
- h : The total hidden dimension of the model.
- a : The number of attention heads.

For each head i (where $i \in \{1, \dots, a\}$), the input matrices are the Query (Q_i), Key (K_i), and Value (V_i).

- The dimension of each head is given by $d_{head} = h/a$.
- The shapes of the matrices for a single head are:

$$Q_i \in \mathbb{R}^{s \times (h/a)}, \quad K_i \in \mathbb{R}^{s \times (h/a)}, \quad V_i \in \mathbb{R}^{s \times (h/a)}$$

The attention output Z_i for head i is calculated using the scaled dot-product formula:

$$Z_i = \text{softmax} \left(\frac{Q_i K_i^T}{\sqrt{h/a}} \right) V_i$$

Assume that multiplying a matrix of size $M \times N$ by a matrix of size $N \times P$ requires approximately $2MNP$ floating-point operations (accumulating products). Derive the total computational complexity (FLOPs) for all a heads by answering the following steps. Ignore the cost of the scalar division ($\sqrt{h/a}$) and the softmax exponentiation/summation operations, as they are negligible compared to the matrix multiplications.

A5. Consider a simple two-layer neural network. The network processes a batch of input data samples. We aim to derive the complete backpropagation formulas (gradients) for the weight matrices W_1 and W_2 .

Let the batch size be B . For a single sample index $b \in \{1, \dots, B\}$, the variables are defined as follows:

- **Input:** $x_b \in \mathbb{R}^d$
- **Hidden Layer (Pre-activation):** $h_b \in \mathbb{R}^p$
- **Hidden Layer (Post-activation):** $z_b \in \mathbb{R}^p$
- **Output Prediction:** $\hat{y}_b \in \mathbb{R}^q$
- **Weights:** $W_1 \in \mathbb{R}^{p \times d}$ and $W_2 \in \mathbb{R}^{q \times p}$
- **Activation Function:** $\sigma(\cdot)$ (applied element-wise)

The forward propagation for a specific sample b is given by:

$$h_b = W_1 x_b \tag{1}$$

$$z_b = \sigma(h_b) \tag{2}$$

$$\hat{y}_b = W_2 z_b \tag{3}$$

The total objective function (loss) f is the average of the loss L over the batch:

$$f = \frac{1}{B} \sum_{b=1}^B L(\hat{y}_b)$$

Using the chain rule in the backward process, derive the expressions of the (stochastic) gradients for each layer.